Estimating Mean Score Reliability with RMU

Giacomo Bignardi

2025-10-02

Introduction

> **i** Note
>
> This tutorial can be run and tested inside the docker image `bignardig/tidyverse461:v6`, which has all the required R packages pre-installed (apart from `cmdstan` itself – see Set Up below). The `v6` image differs from the `v5` image used for the paper's analysis scripts only in that it ships the newer version of `gbtoolbox`, in which the `reliability()` function has been renamed `rmu()`. The old name still works (with a deprecation warning), so the analysis code is unaffected.
>
> You don't need docker to follow along, though – you can also run the code the usual way, in your own R installation. The code in Set Up will install all the necessary R packages for you.

This tutorial demonstrates how to use Relative Measurement Uncertainty (RMU) to estimate mean score reliability. See our pre-print for more information on RMU.

This brief tutorial assumes you're relatively familiar with multilevel Bayesian modelling.

Set Up

Installing the required R packages

The code below checks for the R packages this tutorial needs and installs any that are missing. All of them are on CRAN, except for my package of miscellaneous R functions, `gbtoolbox`, which contains the `rmu()` function used to calculate RMU. `gbtoolbox` is installed from github; if you already have an older version installed, which has `reliability()` but no `rmu()`, it will be updated.

```
# Install any missing CRAN packages (does nothing if they are all already
installed)
required_packages = c("tidyverse", "brms", "ggdist", "psych", "remotes")
missing_packages           =      required_packages[!sapply(required_packages,
requireNamespace, quietly = TRUE)]
if (length(missing_packages) > 0) install.packages(missing_packages)
```

```r
# Install (or update) gbtoolbox from github if rmu() isn't available
if (!requireNamespace("gbtoolbox", quietly = TRUE) || !"rmu" %in%
getNamespaceExports("gbtoolbox")) {
  remotes::install_github("giac01/gbtoolbox")
}
```

! Installing Stan (`cmdstanr` or `rstan`)

`brms` fits models using Stan, which needs to be installed separately from the R packages above. Two R interfaces ("backends") to Stan are available:

- **`cmdstanr`** (used in this tutorial, via `backend = "cmdstanr"`) is not on CRAN. Install the R package from the Stan R-universe repository, then install `cmdstan` itself, which needs a working C++ toolchain:

  ```r
  install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev",
  getOption("repos")))
  cmdstanr::check_cmdstan_toolchain(fix = TRUE)
  cmdstanr::install_cmdstan()
  ```

  See the cmdstanr installation guide if you run into problems.

- **`rstan`** is on CRAN and is installed automatically alongside `brms`, but also requires a C++ toolchain – see the rstan installation guide. It is `brms`'s default backend. If you'd rather use it, delete the `backend = "cmdstanr"` and `threads = threading(...)` arguments from the `brm()` calls below. (The `threads` argument splits each chain's likelihood computation across several CPU threads; `static = TRUE` makes that split deterministic, so that the results are exactly reproducible given the `seed`.)

Simulating data

Imagine we measure the length (in cm) of 5000 mice 5 times each. Every time we measure a mouse, we will get a slightly different measurement (have you tried holding a mouse with one hand and holding a ruler in another?).

In this example, imagine that each measurement from each subject follows a normal distribution with a fixed mean (called the "true score" in psychometrics; θ_i) and fixed variance (σ^2_ϵ). Assume that this measurement error variance (σ^2_ϵ) is the same across all participants.

The final important parameter to consider is the variance in mouse true scores across the mouse population (σ^2_θ).

To illustrate, let's simulate some data:

```
library(ggdist)
library(brms)
library(cmdstanr)
library(psych)
library(tidyverse)
library(gbtoolbox)

rm(list=ls())
set.seed(1)
N                  = 5000 # number of subjects (mice)
J                  = 5   # number of measurements per subject
true_score_variance = 3
error_variance      = 2

df = expand.grid(j = 1:J, mouse = 1:N)

true_scores       = rnorm(N, mean = 10, sd = sqrt(true_score_variance))
measurement_error = rnorm(N*J, mean = 0, sd = sqrt(error_variance))

df$measurement = true_scores[df$mouse] + measurement_error

df_average_lengths = df |>
  group_by(mouse) |>
  summarise(average_measurement = mean(measurement))
```

In the above example, we can work out the reliability of the mean score using J, σ_ϵ^2 and σ_θ^2:

$$\begin{aligned} \mathrm{R}\left(\hat{\mathbf{y}}\right) &= \frac{\sigma_\theta^2}{\sigma_\theta^2 + \sigma_\epsilon^2 / \mathrm{J}} \\ &= \frac{3}{3 + 2/5} = 0.882 \end{aligned}$$

In case you don't trust me, the derivation for this formula is in the appendix. However, we can also check it's correct by computing the squared correlation between the observed mean scores and true scores:

```
cor(df_average_lengths$average_measurement, true_scores)^2
```

```
[1] 0.8902067
```

Estimating Reliability

It's worth noting that the Bayesian measurement model we employ in this example effectively makes the same assumptions as coefficient alpha - which if violated can lead to bias (this paper covers the assumptions of alpha nicely).

However, with RMU, we can make our model more complex (e.g., use an alternative outcome distribution, add parameters, etc.) to account for any quirks with our measurements. For example, for modelling reaction times, you may want to use a more appropriate distribution.

We will keep our model simple in this example - but if you want to apply this code to real data - it's worth considering whether the model assumptions are met!

(1) Coefficient alpha

We can estimate reliability using (unstandardised) coefficient alpha. First, we need to pivot_wider our dataset so that each row represents a mouse and our five measurements are on five columns:

```
df_wide = df |>
  pivot_wider(
    id_cols = mouse,
    names_from = j,
    values_from = measurement,
    names_prefix = "measurement"
  )

df_wide |>
  head()
```

```
# A tibble: 6 × 6
  mouse measurement1 measurement2 measurement3 measurement4 measurement5
  <int>        <dbl>        <dbl>        <dbl>        <dbl>        <dbl>
1     1         6.77         9.80         6.54        10.6          10.5
2     2         8.57         8.58        11.2         10.7          10.2
3     3         7.41         8.71         8.30         8.63         10.4
4     4        12.1         11.6         12.0         12.4          11.9
5     5        11.1         12.6         12.5         11.2          10.2
6     6         6.54         7.19         6.89         8.49         11.3
```

Coefficient alpha calc also gives us .88!

```
alpha = psych::alpha(df_wide[-1])$total$raw_alpha

alpha_ci = psych::alpha.ci(alpha = alpha, n.obs = N, n.var = J, digits = 4)

alpha_ci
```

```
   95% confidence boundaries (Feldt)
 lower alpha upper
  0.88  0.89  0.89
```

(2) RMU

To calculate RMU, we need to first fit a Bayesian measurement model, and then extract a matrix of posterior draws for each mouse's length parameter.

We can use a simple random intercept multilevel model, as shown below.

To fit the model, we use a "long" dataset which has a column for measurement, and a column to indicate mouse. Note that multiple measurements from each mouse are stored in the same column (measurement).

```
head(df)
```

```
  j mouse measurement
1 1     1    6.770474
2 2     1    9.804690
3 3     1    6.541625
4 4     1   10.583413
5 5     1   10.495552
6 1     2    8.567657
```

Fit brms Multilevel Model

Let's fit our model:

```
brms_model = brm(
  measurement ~ 1 + (1 | mouse),
  data    = df,
  chains  = 4,
  cores   = 4,
  seed    = 1,           # So that the results are reproducible
  backend = "cmdstanr",  # This is to speed up model fitting using parallelization
   threads = threading(2, static = TRUE) # Also for speed; static = TRUE keeps
the results reproducible
  )
```

And look at our results:

```
print(summary(brms_model), digits = 4)
```

```
 Family: gaussian
  Links: mu = identity
Formula: measurement ~ 1 + (1 | mouse)
   Data: df (Number of observations: 25000)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Multilevel Hyperparameters:
~mouse (Number of levels: 5000)
              Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
sd(Intercept)   1.7650    0.0198   1.7274   1.8040 1.0022     1023     1668

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
Intercept   9.9928    0.0252   9.9444  10.0422 1.0044      880     1936

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
sigma   1.4185    0.0070   1.4049   1.4323 0.9996     3906     2789

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Our fitted model has the following structure, where i indexes mice and j indexes measurements (as in the appendix), and the estimates are rounded to whole numbers:

$$
\begin{aligned}
measurement_{ij} &= 10 + \theta_i + \epsilon_{ij} \\
\theta_i &\sim N(0, \sigma_\theta^2 = 3) \quad \text{(mouse random-intercepts)} \\
\epsilon_{ij} &\sim N(0, \sigma_\epsilon^2 = 2) \quad \text{(measurement error)}
\end{aligned}
$$

Note that the normal distributions above are written in terms of *variances*, whereas the `brms` summary reports *standard deviations*: `sd(Intercept)` is the estimate of $\sigma_\theta = \sqrt{3} = 1.73$, and `sigma` is the estimate of $\sigma_\epsilon = \sqrt{2} = 1.41$.

Visualising the posteriors

Before calculating RMU, it helps to look at what it is calculated from. The function below plots the posterior density of the random intercept (θ_i) for a random sample of 200 mice, with one curve per mouse. We wrap it in a function because we will reuse it for the other models fitted later in this tutorial.

```
plot_posterior_densities = function(brms_model, n_subjects = 200, seed = 1){
  set.seed(seed)
```

```r
  draws = brms_model |>
    as_draws_df() |>
    select(starts_with("r_mouse"))

  draws |>
     select(sample(ncol(draws), n_subjects)) |>                # random sample
of mice
    pivot_longer(everything(), names_to = "mouse", values_to = "draw") |>
    ggplot(aes(x = draw, group = mouse)) +
     geom_line(stat = "density", colour = "steelblue", alpha = .3, linewidth
= .3) +
    labs(
      x        = "Mouse random intercept (posterior draws)",
      y        = "Posterior density",
      subtitle = paste("Posterior densities for", n_subjects, "randomly sampled
mice")
    ) +
    theme_minimal()
}

plot_posterior_densities(brms_model)
```



Each mouse's posterior is fairly narrow compared to how spread out the mice are from each other, which is why reliability is high here. This is essentially what RMU quantifies: the variance between the subjects' posteriors relative to the uncertainty within them.

Calculate RMU

RMU is calculated from the posterior draws of each mouse's random intercept parameter, using the `rmu()` function from `gbtoolbox` (loaded in Set Up). `rmu()` can be used in two ways: you can pass it the fitted `brms` model directly and let it extract the posterior draws for you, or you can extract a matrix of posterior draws yourself and pass that. Both are shown below.

Option 1: Pass the fitted brms model directly

If you pass a fitted `brms` model (an object of class `brmsfit`) to `rmu()`, it extracts the posterior draws for every group-level ("random") effect in the model, and calculates RMU for each one. Our model only has one, the random intercept for `mouse`, so we get a single result, named `mouse__Intercept` (following the `sd_mouse__Intercept` naming that `brms` itself uses):

```
set.seed(1)

rmu_brms = rmu(brms_model)
```

```
Warning: rmu() for brmsfit objects is new and has not yet been tested
extensively. Please check the output carefully and report any bugs at
https://github.com/giac01/gbtoolbox/issues (this warning is shown once per
session)
```

```

Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of levels per grouping factor: mouse = 5000
  Number of input posterior draws:      4000
  Number of RMU posterior draws:        2000

                rmu_estimate hdci_lowerbound hdci_upperbound
mouse__Intercept        0.886           0.881            0.89
```

The function prints a summary table, and (invisibly) returns a list with one element per group-level effect. Each element contains a data frame `hdci` with the point estimate and highest density credible interval, and the vector of RMU posterior draws, `reliability_posterior_draws`:

```
rmu_brms$mouse__Intercept$hdci
```

```
  rmu_estimate hdci_lowerbound hdci_upperbound .width .point .interval
1    0.8855564       0.8807095       0.8903484   0.95   mean      hdci
```

Two things to note here. First, the method for `brms` models is new, so it currently prints a warning (once per R session) asking you to check its output carefully – we do exactly that below. Second, models with several grouping factors (e.g. `(1 | mouse) + (1 | measurement_occasion)`) or several group-level coefficients (e.g. `(1 + x | mouse)`) will return one RMU estimate per coefficient per grouping factor. `rmu()` can't tell which grouping factor represents your subjects, so use the `group` and `coef` arguments to pick out the ones you want (e.g. `rmu(brms_model, group = "mouse", coef = "Intercept")`).

Option 2: Extract the posterior draws yourself

Alternatively, and this is what `rmu()` does for you under the hood, extract the posterior draws for each mouse's random intercept into a matrix with **subjects on the rows and posterior draws on the columns**, and pass that matrix to `rmu()`. This approach works for any Bayesian model, not just those fit with `brms`, as long as you can get hold of the posterior draws for each subject's parameter:

```
set.seed(1)

posterior_draws = brms_model |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  t()

nrow(posterior_draws) # should equal the number of subjects
```

```
[1] 5000
```

```
ncol(posterior_draws) # should equal the number of post-warmup posterior draws
```

```
[1] 4000
```

```
rmu(posterior_draws)
```

```
Reliability (Relative Measurement Uncertainty)
  Number of subjects:              5000
  Number of input posterior draws: 4000
  Number of RMU posterior draws:   2000
```

```
  RMU estimate: 0.886
  Posterior SD: 0.003
  95% HDCI:     [0.881, 0.890]

Access full results with $hdci and $reliability_posterior_draws
```

Remember to check that the matrix of draws has subjects on the rows and draws on the columns. We can check this because we know we have 5000 subjects, and the function has confirmed this is the case.

Both options give identical results here. They use exactly the same posterior draws, and because `rmu()` randomly splits the posterior draws in half when calculating RMU (see the go/no-go tutorial for the details), we set the same seed before each call to get exactly the same random split. Without setting a seed, the two results would still be very similar, but not identical.

(3) Test-retest reliability of the posterior means

Because we simulated the data ourselves, we know what the reliability of the mean scores should be (0.882), and we can check it against the true scores. With real data we have neither. The usual way to check the reliability of a score empirically is then *test-retest*: measure the same subjects a second time, and correlate the two sets of scores. We can do exactly the same thing with our Bayesian scores, where each mouse's score is the posterior mean of its random intercept, θ_i.

Let's simulate a second, independent, round of five measurements of the same mice (same true scores, but new measurement errors), and fit the same model again to the new data:

```
set.seed(2)

df_retest             = df
df_retest$measurement = true_scores[df_retest$mouse] + rnorm(N*J, mean = 0, sd
= sqrt(error_variance))

brms_model_retest = brm(
  measurement ~ 1 + (1 | mouse),
  data    = df_retest,
  chains  = 4,
  cores   = 4,
  seed    = 2,
  backend = "cmdstanr",
  threads = threading(2, static = TRUE)
  )
```

We then extract each mouse's posterior mean from both fitted models. Note that these posterior means are shrunk towards the population mean, as is usual for random intercepts, but in a

balanced design like this one the shrinkage is the same for every mouse, so it does not affect the correlations below:

```
posterior_means_test = brms_model |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  colMeans()

posterior_means_retest = brms_model_retest |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  colMeans()
```

As a quick check of that last point, each mouse's posterior mean should be (almost) a linear function of its mean score across the five measurements, so the two should be near-perfectly correlated:

```
cor(df_average_lengths$average_measurement, posterior_means_test)
```

```
[1] 0.9999926
```

The correlation between the two sets of posterior means is the test-retest reliability of our Bayesian scores. Let's compare it with everything else we have calculated so far:

```
data.frame(
  quantity = c(
    "Reliability from the formula",
    "Coefficient alpha",
    "RMU",
    "Test-retest correlation of posterior means",
    "Squared cor. between posterior means and true scores"
  ),
  estimate = c(
    true_score_variance/(true_score_variance+error_variance/J),
    alpha,
    rmu_brms$mouse__Intercept$hdci$rmu_estimate,
    cor(posterior_means_test, posterior_means_retest),
    cor(posterior_means_test, true_scores)^2
  ) |> round(3)
)
```

```
                                          quantity estimate
1                        Reliability from the formula    0.882
2                                  Coefficient alpha    0.886
3                                                RMU    0.886
4          Test-retest correlation of posterior means    0.891
5 Squared cor. between posterior means and true scores    0.890
```

They all agree. The key difference is what each one needs: the formula needs the simulation parameters, the squared correlation with the true scores needs the true scores (which we never have in practice), and the test-retest correlation needs every mouse to be measured twice. RMU gets to the same answer from a single round of measurements.

(4) Comparing the mean scores and the posterior means

We have seen that the mean scores and the posterior means are almost perfectly correlated, but they are not the same numbers. Let's compare their means and variances:

```
data.frame(
  score    = c("Mean score", "Posterior mean of random intercept"),
       mean                =   c(mean(df_average_lengths$average_measurement),
mean(posterior_means_test)),
            variance      =     c(var(df_average_lengths$average_measurement),
var(posterior_means_test))
) |>
  mutate(across(c(mean, variance), \(x) round(x, 3)))
```

```
                            score  mean variance
1                      Mean score 9.994    3.518
2 Posterior mean of random intercept 0.001    2.759
```

The means differ simply because the random intercepts are *deviations* from the population intercept (around 10cm). Adding the model's intercept to each mouse's posterior mean puts them back on the same scale as the mean scores.

The variances are more interesting. From the simulation, the true scores have a variance of 3.

The variance of the mean scores is *larger* than this, because each mean score still contains some measurement error: we expect $\sigma^2_\theta + \sigma^2_\epsilon / \ \mathrm{J} = 3 + 2/5 = 3.4$.

The variance of the posterior means is *smaller* than 3, because each posterior mean is shrunk towards the population mean.

(The observed variance of the mean scores is a little above 3.4 because, by chance, the true score variance in our simulated sample of mice came out slightly above 3: the model estimates it at about 3.1.)

This gives us another way of looking at reliability: the variance of the posterior means, relative to the model's estimate of the true score variance (the squared `sd(Intercept)` from the model summary), is approximately equal to the reliability:

```
true_score_variance_estimate    =    VarCorr(brms_model)$mouse$sd["Intercept",
"Estimate"]^2

var(posterior_means_test) / true_score_variance_estimate
```

```
[1] 0.8856484
```

Two things to take away from this. First, the variance of the posterior means is *not* an estimate of the true score variance – it is deflated by shrinkage – so if you want the true score variance, use the model's `sd(Intercept)` estimate. Second, the posterior means are only shrunk by the same amount for every mouse in a balanced design. When the number of measurements differs between mice, as in the censored example below, mice with fewer measurements are shrunk more, and the posterior means are no longer just a rescaled version of the mean scores.

Recap of simulation code

Just to check this wasn't a fluke, we can play around with the parameters J and true_score_variance and error_variance below to show that we get the correct result across different simulation conditions:

```
rm(list = setdiff(ls(), "plot_posterior_densities")) # clear everything except
our plotting function
set.seed(1)
N                  = 5000 # number of subjects (mice)
J                  = 3    # number of measurements per subject
true_score_variance = 1
error_variance      = 10

df = expand.grid(j = 1:J, mouse = 1:N)

true_scores       = rnorm(N, mean = 10, sd = sqrt(true_score_variance))
measurement_error = rnorm(N*J, mean = 0, sd = sqrt(error_variance))

df$measurement = true_scores[df$mouse] + measurement_error

df_average_lengths = df |>
  group_by(mouse) |>
  summarise(average_measurement = mean(measurement))
```

```
# Fit model and calculate RMU
# I've reduced the number of MCMC chains here to two for speed - you may want
more in practice!

brms_model = brm(
  measurement ~ 1 + (1 | mouse),
  data    = df,
  chains  = 2,
  cores   = 2,
  seed    = 1,
  refresh = 0,           # This hides the progress bar
 backend = "cmdstanr",  # This is to speed up model fitting using parallelization
  threads = threading(4, static = TRUE) # Also for speed; static = TRUE keeps
the results reproducible
  )

set.seed(1)
rmu_recap = rmu(brms_model)
```

For the above example, RMU is pretty similar to the expected value:

```
cat("Simulation reliability = ")
```

```
Simulation reliability =
```

```
true_score_variance/(true_score_variance+error_variance/J)
```

```
[1] 0.2307692
```

```
cat("Squared Correlation between mean and true scores  = ")
```

```
Squared Correlation between mean and true scores  =
```

```
# This may slightly differ to above due to sampling variance

cor(df_average_lengths$average_measurement, true_scores)^2
```

```
[1] 0.2414736
```

```
cat("RMU:\n")
```

```
RMU:
```

```
rmu_recap
```

```
Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of levels per grouping factor: mouse = 5000
  Number of input posterior draws:      2000
  Number of RMU posterior draws:        1000

                 rmu_estimate hdci_lowerbound hdci_upperbound
mouse__Intercept         0.25           0.217           0.285
```

With only three measurements per mouse and a much larger error variance, the posteriors are now wide relative to the spread between mice, so the curves overlap heavily. Compare this plot to the one from our first model:

```
plot_posterior_densities(brms_model)
```



Beyond coefficient alpha

We can leverage the flexibility of brms + RMU to estimate reliability in more complex scenarios.

We will update the simulation code from our first example. This time our mice are a little larger, with a mean true score of 13cm rather than 10cm, and our ruler is only 15cm long, so that some mice are too long to measure. We also make the following two adjustments:

1. For each mouse, we randomly draw a number between 2 and 5 for the number of measurements.
2. Measurements greater than 15cm (the length of the ruler) are recoded as ">15cm". In other words, these measurements are *right-censored*: we know the mouse is longer than 15cm, but not by how much.

To store the censored measurements in a form that `brms` understands, we keep two columns. The `measurement` column holds the ruler reading, with a reading of ">15cm" recorded as the censoring point (15). The `censored` column indicates whether each reading was observed exactly (`"none"`) or censored (`"right"`). We wrap the simulation in a function, because we will reuse it below to simulate a second round of measurements of the same mice.

```
rm(list = setdiff(ls(), "plot_posterior_densities")) # clear everything except
our plotting function
set.seed(1)
N                  = 5000 # number of subjects (mice)
true_score_variance = 3
error_variance      = 2
ruler_length        = 15   # measurements above this are censored

true_scores = rnorm(N, mean = 13, sd = sqrt(true_score_variance)) # slightly
larger mice than before

# Simulate one round of measurements for every mouse
simulate_measurements = function(true_scores){
  N  = length(true_scores)
  J  = sample(2:5, size = N, replace = TRUE) # number of measurements per mouse

  df = data.frame(
    mouse = rep(1:N, times = J),
    j     = sequence(J)
  )

   df$measurement  =  true_scores[df$mouse]  +  rnorm(nrow(df),  mean  =  0,  sd  =
sqrt(error_variance))
   df$censored     = ifelse(df$measurement > ruler_length, "right", "none") #
">15cm" readings
```

```
  df$measurement = pmin(df$measurement, ruler_length)                    # are
recorded as 15

  return(df)
}

df = simulate_measurements(true_scores)

head(df, 10)
```

```
   mouse j measurement censored
1      1 1    10.79093     none
2      1 2    12.77208     none
3      2 1    11.81552     none
4      2 2    14.75945     none
5      2 3    13.56926     none
6      2 4    11.86006     none
7      3 1    13.15446     none
8      3 2    11.60971     none
9      3 3    12.39679     none
10     4 1    13.88276     none
```

The number of measurements now varies between mice:

```
df |>
  count(mouse, name = "n_measurements") |>
  count(n_measurements, name = "n_mice")
```

```
  n_measurements n_mice
1              2   1269
2              3   1174
3              4   1281
4              5   1276
```

And 3290 of the 17564 measurements (18.7%) were censored at “>15cm”:

```
df |>
  filter(censored == "right") |>
  head()
```

```
  mouse j measurement censored
1     4 2          15    right
2     4 3          15    right
3     4 4          15    right
4     7 3          15    right
5     9 3          15    right
6     9 4          15    right
```

This scenario shows where our approach can shine. Coefficient alpha cannot be computed here, as it assumes that every mouse is measured five times (pivoting the data to wide format, as we did above, would leave most mice with missing columns). Alpha also assumes that we have continuous measurements, whereas in this case our measurements are censored. With RMU, we can simply update our model to account for censoring in the outcome variable, which is relatively straightforward in `brms`: we add `| cens(censored)` to the left-hand side of the model formula. For the censored rows, `brms` then replaces the normal likelihood with the probability that the measurement exceeds 15cm (the upper tail of the normal distribution), rather than treating them as if the mouse were exactly 15cm long.

```
brms_model_censored = brm(
  measurement | cens(censored) ~ 1 + (1 | mouse),
  data    = df,
  chains  = 4,
  cores   = 4,
  seed    = 1,
  backend = "cmdstanr",
  threads = threading(2, static = TRUE)
)
```

Despite the censoring and the uneven number of measurements per mouse, the model recovers the simulation parameters nicely: the estimated true score SD is close to $\sqrt{3} = 1.73$ and the residual (measurement error) SD is close to $\sqrt{2} = 1.41$.

```
print(summary(brms_model_censored), digits = 4)
```

```
 Family: gaussian
  Links: mu = identity
Formula: measurement | cens(censored) ~ 1 + (1 | mouse)
   Data: df (Number of observations: 17564)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Multilevel Hyperparameters:
```

```
~mouse (Number of levels: 5000)
              Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
sd(Intercept)   1.7843    0.0229   1.7404   1.8292 1.0010     1404     2162

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
Intercept  13.0023    0.0277  12.9495  13.0564 1.0005     1407     2302

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI   Rhat Bulk_ESS Tail_ESS
sigma   1.4058    0.0099   1.3864   1.4250 0.9998     2631     2514

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

RMU is calculated from the fitted model exactly as before:

```
set.seed(1)
rmu_censored = rmu(brms_model_censored)
```

```


Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of levels per grouping factor: mouse = 5000
  Number of input posterior draws:      4000
  Number of RMU posterior draws:        2000

                 rmu_estimate hdci_lowerbound hdci_upperbound
mouse__Intercept         0.82           0.813           0.828
```

The posteriors now vary in width between mice, because each mouse was measured a different number of times and, for the longer mice, some of those measurements were censored:

```
plot_posterior_densities(brms_model_censored)
```



Test-retest reliability of the posterior mean estimates

In our first example, the balanced design meant that we could work out the expected reliability of the mean scores from a simple formula. That formula no longer applies here, because each mouse is measured a different number of times (and some measurements are censored). Instead, we can find the "true" reliability of our mouse length estimates empirically, using the same test-retest approach as before: we measure the same mice a second time.

As before, we simulate a second, independent, round of measurements of the same mice (same true scores, but this time also a new number of measurements for each mouse, as well as new measurement errors), and fit the same model again:

```
set.seed(2)

df_retest = simulate_measurements(true_scores)

brms_model_censored_retest = brm(
  measurement | cens(censored) ~ 1 + (1 | mouse),
  data    = df_retest,
  chains  = 4,
  cores   = 4,
  seed    = 2,
  backend = "cmdstanr",
  threads = threading(2, static = TRUE)
)
```

```
set.seed(1)
rmu_censored_retest = rmu(brms_model_censored_retest)
```

We then extract the posterior mean of each mouse's random intercept from both fitted models. The correlation between the two sets of posterior means is the test-retest reliability of our estimates, and should match RMU. We can also check the squared correlation between the posterior means and the true scores, as we did before:

```
posterior_means_test = brms_model_censored |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  colMeans()

posterior_means_retest = brms_model_censored_retest |>
  as_draws_df() |>
  select(starts_with("r_mouse")) |>
  colMeans()

data.frame(
  quantity = c(
    "RMU, first round of measurements",
    "RMU, second round of measurements",
    "Test-retest correlation of posterior means",
    "Squared cor. with true scores, first round",
    "Squared cor. with true scores, second round"
  ),
  estimate = c(
    rmu_censored$mouse__Intercept$hdci$rmu_estimate,
    rmu_censored_retest$mouse__Intercept$hdci$rmu_estimate,
    cor(posterior_means_test, posterior_means_retest),
    cor(posterior_means_test, true_scores)^2,
    cor(posterior_means_retest, true_scores)^2
  ) |> round(3)
)
```

```
                                    quantity estimate
1            RMU, first round of measurements    0.820
2           RMU, second round of measurements    0.821
3  Test-retest correlation of posterior means    0.830
4  Squared cor. with true scores, first round    0.823
5 Squared cor. with true scores, second round    0.817
```

The RMU estimates from both rounds of measurements closely match the test-retest correlation of the posterior means, and the squared correlation between the posterior means and the true scores. In other words, RMU recovers the reliability of our estimates from a *single* round of measurements, without the need to measure every mouse twice, and without assuming that every mouse was measured the same number of times with an infinitely long ruler.

Appendix: Deriving The Mean Score Reliability Formula

Consider a measurement model where we measure N participants, and we measure each participant $\mathcal{J}$ times (or across J items). Each participant has a true score θ_i which represents the *expected* measurement for participant i.

We assume that measurements from a given subject follow a normal distribution with mean (θ_i) and variance (σ^2_ϵ).

We assume that the distribution of true scores across the population is normally distributed with mean μ and variance σ^2_θ.

We can describe the distribution of each measurement as follows:

$$\overbrace{y_{ij}}^{\text{measurement } j \text{ for subject } i} = \overbrace{\theta_i}^{\text{subject } i \text{ true score}} + \overbrace{\epsilon_{ij}}^{\text{measurement error}}$$

$$\epsilon_{ij} \sim \text{Normal}\ (0, \overbrace{\sigma^2_\epsilon}^{\text{error variance}})$$

$$\overbrace{\theta_1, \theta_2, ..., \theta_N}^{\text{population true scores}} \sim \text{Normal}\ (\overbrace{\mu}^{\text{population mean true score}}, \overbrace{\sigma^2_\theta}^{\text{true score variance}})$$

Calculating Reliability of Mean Scores

Given that we know the distribution of each measurement, the distribution of a mean of multiple independent measurements is also Normal but with a smaller variance (note that as J increases, the variance decreases):

$$\begin{aligned}
\hat{y}_i &= \frac{1}{\text{J}} \sum_{j=1}^{\text{J}} y_{ij} \\
&= \frac{1}{\text{J}} \sum_{j=1}^{\text{J}} (\theta_i + \epsilon_{ij}) \\
&= \theta_i + \sum_{j=1}^{\text{J}} \frac{\epsilon_{ij}}{\text{J}} \\
&= \theta_i + \mathcal{E}_i \\
\mathcal{E}_i &= \frac{\epsilon_{i1} + \epsilon_{i2} + ... + \epsilon_{i\ \text{J}}}{\text{J}} \sim \text{Normal}\ (0, \frac{\sigma^2_\epsilon}{\text{J}})
\end{aligned}$$

To work out the variance of the mean score for subject i, note that the variance of the sum of J independent normally distributed variables ($\epsilon_{i1} + \epsilon_{i2} + ... + \epsilon_{i\ \mathrm{J}}$) is equal to J σ_ϵ^2, and dividing by a constant J divides the variance by J^2.

Reliability Formula

Reliability is the ratio of true score variance divided by the observed score variance:

$$\begin{aligned} \mathrm{R}\ (\hat{\mathbf{y}}) &= \mathrm{R}\ (\boldsymbol{\theta} + \mathcal{E}) \\ &= \frac{\mathrm{Var}\ (\boldsymbol{\theta})}{\mathrm{Var}\ (\boldsymbol{\theta}) + \mathrm{Var}\ (\mathcal{E})} \\ &= \frac{\sigma_\theta^2}{\sigma_\theta^2 + \sigma_\epsilon^2 / \ \mathrm{J}} \end{aligned}$$

This formula shows that reliability increases as we add more items (J increases), because the error variance gets divided by J.

Session Info

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.4 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK:    /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
 [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
 [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
 [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
 [9] LC_ADDRESS=C               LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C

time zone: Etc/UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base
```

```
other attached packages:
 [1] gbtoolbox_0.0.4 lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
 [5] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
 [9] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0 psych_2.6.5
[13] cmdstanr_0.9.0  brms_2.23.0     Rcpp_1.1.2      ggdist_3.3.3

loaded via a namespace (and not attached):
 [1] gtable_0.3.6          tensorA_0.36.2.1      QuickJSR_1.10.0
 [4] xfun_0.59             remotes_2.5.0         processx_3.9.0
 [7] inline_0.3.21         lattice_0.22-9        tzdb_0.5.0
[10] vctrs_0.7.3           tools_4.6.1           ps_1.9.3
[13] generics_0.1.4        curl_7.1.0            stats4_4.6.1
[16] parallel_4.6.1        pkgconfig_2.0.3       Matrix_1.7-5
[19] data.table_1.18.4     checkmate_2.3.4       RColorBrewer_1.1-3
[22] S7_0.2.2              distributional_0.8.1  RcppParallel_5.1.11-2
[25] lifecycle_1.0.5       compiler_4.6.1        farver_2.1.2
[28] Brobdingnag_1.2-9     mnormt_2.1.2          codetools_0.2-20
[31] htmltools_0.5.9       bayesplot_1.15.0      yaml_2.3.12
[34] pillar_1.11.1         StanHeaders_2.32.10   bridgesampling_1.2-1
[37] abind_1.4-8           nlme_3.1-169          rstan_2.32.7
[40] posterior_1.7.0       tidyselect_1.2.1      digest_0.6.39
[43] mvtnorm_1.4-1         stringi_1.8.7         reshape2_1.4.5
[46] labeling_0.4.3        fastmap_1.2.0         grid_4.6.1
[49] cli_3.6.6             magrittr_2.0.5        loo_2.10.0
[52] pkgbuild_1.4.8        withr_3.0.3           scales_1.4.0
[55] backports_1.5.1       timechange_0.4.0      rmarkdown_2.31
[58] matrixStats_1.5.0     otel_0.2.0            gridExtra_2.3.1
[61] hms_1.1.4             coda_0.19-4.1         evaluate_1.0.5
[64] knitr_1.51            V8_8.2.0              rstantools_2.6.0
[67] rlang_1.3.0           glue_1.8.1            jsonlite_2.0.0
[70] plyr_1.8.9            R6_2.6.1
```